OpenCV Notes (Beginner Friendly)

1. What is OpenCV?

OpenCV (Open Source Computer Vision Library) is a free library used for:

- Image Processing
- Computer Vision
- Video Analysis
- Face Detection
- Object Detection
- Motion Tracking
- AI and Machine Learning projects

OpenCV works with Python, C++, Java, and many other programming languages.

---

2. Features of OpenCV

- Image reading and writing
- Video capture
- Face detection
- Object detection
- Color detection
- Image filtering
- Edge detection
- Shape recognition
- Barcode and QR code detection

---

3. Installing OpenCV

Install OpenCV using pip:

pip install opencv-python

For extra modules:

pip install opencv-contrib-python

Import OpenCV:

import cv2

---

4. Reading an Image

import cv2

image = cv2.imread("image.jpg")

cv2.imshow("Image", image)

cv2.waitKey(0)

cv2.destroyAllWindows()

Explanation

- imread() → Reads an image.
- imshow() → Displays the image.
- waitKey(0) → Waits until a key is pressed.
- destroyAllWindows() → Closes all windows.

---

5. Saving an Image

cv2.imwrite("new_image.jpg", image)

6. Reading a Video

import cv2

video = cv2.VideoCapture("video.mp4")

while True:

ret, frame = video.read()

if not ret:

break

```
cv2.imshow("Video", frame)
if cv2.waitKey(1) == ord('q'):
        break
video.release()
cv2.destroyAllWindows()
```

---

7. Webcam Capture

```
import cv2
camera = cv2.VideoCapture(0)
while True:
    ret, frame = camera.read()
cv2.imshow("Webcam", frame)
if cv2.waitKey(1) == ord('q'):
        break
camera.release()
cv2.destroyAllWindows()
```

---

8. Image Properties

```
import cv2
img = cv2.imread("image.jpg")
print(img.shape)
print(img.size)
print(img.dtype)
```

Output

- Shape → Height, Width, Channels
- Size → Total pixels

- Data Type → uint8

---

9. Resize an Image

```
resized = cv2.resize(img, (400,300))
cv2.imshow("Resized", resized)
cv2.waitKey(0)
```

---

10. Crop an Image

```
crop = img[100:300,100:300]
cv2.imshow("Crop", crop)
cv2.waitKey(0)
```

---

11. Convert to Grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow("Gray", gray)
```

---

12. Blur an Image

```
blur = cv2.GaussianBlur(img,(5,5),0)
cv2.imshow("Blur", blur)
```

---

13. Edge Detection (Canny)

```
edges = cv2.Canny(img,100,200)
cv2.imshow("Edges", edges)
```

---

14. Drawing Shapes

Draw Line

```
cv2.line(img,(50,50),(250,50),(0,255,0),3)
```

Draw Rectangle

```
cv2.rectangle(img,(50,50),(250,200),(255,0,0),2)
```

Draw Circle

```
cv2.circle(img,(150,150),50,(0,0,255),3)
```

---

15. Put Text

```
cv2.putText(
    img,
    "OpenCV",
    (50,50),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (255,255,255),
    2
)
```

---

16. Face Detection

```
import cv2

face = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml"
)

img = cv2.imread("person.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = face.detectMultiScale(gray,1.3,5)

for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),2)

cv2.imshow("Face Detection", img)
cv2.waitKey(0)
```

---

17. Color Detection

```
import cv2

img = cv2.imread("image.jpg")

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

HSV stands for:

- H → Hue
- S → Saturation
- V → Value

---

18. Image Thresholding

```
gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

_,thresh = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)
```

```
cv2.imshow("Threshold",thresh)
```

---

19. Contour Detection

```
contours, hierarchy = cv2.findContours(
   thresh,
   cv2.RETR_TREE,
   cv2.CHAIN_APPROX_SIMPLE
)
cv2.drawContours(img,contours,-1,(0,255,0),2)
```

---

20. Image Rotation

```
cv2.rotate(img,cv2.ROTATE_90_CLOCKWISE)
```

---

21. Flip an Image

```
flip = cv2.flip(img,1)
```

- 0 → Vertical
- 1 → Horizontal
- -1 → Both

---

22. Bitwise Operations

```
cv2.bitwise_and()
cv2.bitwise_or()
cv2.bitwise_not()
cv2.bitwise_xor()
```

Used in masking and image blending.

---

23. Common OpenCV Functions

Function	Purpose
cv2.imread()	Read image
cv2.imwrite()	Save image
cv2.imshow()	Show image
cv2.waitKey()	Wait for key
cv2.destroyAllWindows()	Close windows
cv2.VideoCapture()	Capture webcam/video
cv2.resize()	Resize image
cv2.cvtColor()	Convert color
cv2.GaussianBlur()	Blur image
cv2.Canny()	Detect edges
cv2.line()	Draw line
cv2.rectangle()	Draw rectangle
cv2.circle()	Draw circle
cv2.putText()	Write text
cv2.threshold()	Thresholding
cv2.findContours()	Find contours

Applications of OpenCV

- Face Recognition
- Fingerprint Recognition
- Vehicle Number Plate Recognition (ANPR)
- QR Code Scanner

- Motion Detection
- Traffic Monitoring
- Medical Image Analysis
- Self-driving Cars
- Security and Surveillance
- Robotics
- Augmented Reality (AR)

Advantages

- Free and open source
- Cross-platform (Windows, Linux, macOS)
- Fast image processing
- Large developer community
- Supports Python, C++, Java, and more
- Easy integration with AI and Machine Learning

Limitations

- Advanced computer vision concepts can be complex.
- Some tasks require machine learning models for high accuracy.
- Real-time video processing may require powerful hardware.
